SaltBench: A Referee-Gated Protocol for Measuring Method Effects in Machine-Checked Software Work

Jason Hickey

September 2026

Abstract

SaltBench is a benchmark protocol for one question: How does a machine referee change the way a coding agent works? A machine referee — a proof kernel, a program verifier, or a withheld test suite — decides what an agent's work is worth, and the agent cannot argue with it. Here we report a protocol that makes the referee's effect measurable and whose answers cannot be narrated afterwards: every outcome is decided outside the agent's own toolchain; the agent is walled off from the network, the reference solutions and the harness itself, and the wall is tested by probes that try to breach it before any scored run, so the isolation is observed rather than assumed; every run is authorized by a dated freeze with its predictions registered; and a budget stop is a halt, never a failure. In this study, the subject of the benchmark is a "seat", meaning an agent session in its standard harness. We tested five systems components, all authored in Rust under a pinned Verus toolchain, with a withheld test suite as the referee for each. Four arms are tested: a plain agent; an agent that is also instructed to create a specification and verify the code against it, in a reduced rendering of the method, as registered; and two arms where the specification is provided a priori, which in this matrix ran on one component. We found that the verification arms cost more, and by a practical margin: across these five components no premium exceeded 2.8879× under either reading of the declared set, and the three cheapest sat below 1.4×. That bound is a property of this population and not a promise about larger ones: the premium runs near 1 on the smallest components and rises with size. We publish the complete record.

1 Introduction

An agent's account of its own work is the least reliable evidence about that work. A benchmark that lets the agent grade itself, or lets the experimenter choose the reading after the run, measures the story and not the outcome. SaltBench is built on two refusals. The outcome is decided by a machine referee the agent cannot argue with — a proof kernel, a program verifier, or a withheld test suite, run outside the agent's own toolchain — and the comparison is fixed before the agent runs, so that it cannot be narrated afterwards. The comparison is pre-registered [12]: the population is drawn by a committed seed, the arms are byte-pinned, the predictions are written down, the adverse outcomes are named, and the freeze commit is the authorization. Everything that happens afterwards is a dated amendment appended to the record.

Here we report the benchmark's design and its first measurement, taken end to end on an experiment whose subject is the working seat rather than the model. Section 2.1 states the design and how much of it this paper runs. We claim the protocol; the seat-as-subject design and its

authored population; the cost reading the run returned, together with the qualifiers registered beside it; and a set of instrument findings that we think transfer to any benchmark of this kind. We do not claim an effect of the method under test on what a referee accepts, and we report no magnitude for the population; Section 7 states the tests that could produce one. Three earlier reads on drawn populations are reported in Section 6, compressed to the role they now play, which is the reason this design is shaped as it is. One note on labels, because two of them are numbers: this document is version 1 of the SaltBench report, and *v3* names the campaign generation whose matrix it reports, not an edition of this document.

The method under test, the Salt method [8, 7], is the working discipline the author uses in a formal mathematics project: machine-checked development in which claims travel as kernel-checked artifacts and human attention is reserved for statements and rulings. Its rendering as an arm is described in Section 3.6; only the part of it that a single sealed agent can carry is rendered, and that limit was registered before the first call.

2 The seat-as-subject experiment

The experiment this paper is built around takes the working seat (meaning a coding agent, its harness, and a workspace), not the model, as its subject. A capable agent is handed a systems component to build or to change, under a method it is told to follow, and the question put to it is not whether it can do the work but what the method costs it to do the work. That question forces two things the earlier populations could not supply. The task must be one a referee can decide without consulting the agent's own account of what it did, and the population must be authored rather than drawn, because a drawn population carries a chance, unquantifiable, that the model has already seen the answer.

We present a benchmark design, and an initial measurement over five authored components, run as arms that differ only in the method text the agent is given. Each component is specified by a *card*: the written task the agent is given, carrying its requirements and, where one exists, the formal statement of the task. What the matrix produces is a price per cell, and prices are what this paper reports from it. The cells were scored for cost; the correctness pass taken afterwards, and what it could and could not separate, is reported in Section 4.

2.1 The design, and how much of it this paper runs

The benchmark design has four arms, two-by-two. One axis is the method: a plain agent, against an agent instructed to write a specification and verify its code against it. The other is where the specification comes from: written by the agent, or handed to it a priori as the formal statement of the task. The a priori pair isolates the cost of verifying against a specification from the cost of writing one, and it carries the design's registered gold reading, fixed before any cell ran: if the treatment beats the control when both are handed the statement, the method is the statement. A placebo, an equal-length process prompt with no method content, sits beside the four as a control on prompt length and is not one of them.

The substrate is chosen so that the four arms are comparable. Every arm writes Rust under the same pinned Verus toolchain and is scored by the same withheld suite. Verus is verified Rust, so an arm that writes specifications and proofs and an arm that does not are writing the same language for the same compiler, and the difference between them is the method, not the language.

A fifth arm in which both the code and its specification are written in Lean is a possible extension. It would be comparable to the arms that verify in Rust and Verus, and not to the plain Rust arm, because the language would change together with the method; it is not part of this paper.

Its task forms are four, and they stand at different stages. Greenfield, the agent building a component from its requirements, is the form this paper measures. Brownfield, the agent repairing a component that does not work, is authored and designed and not yet run: every card in the population carries a brownfield rung naming a planted defect, one is realised as a complete tree beside its greenfield twin, and the form's design fixes what a run must record, two verdicts that are never one number and three outcome classes of which only a repaired given file counts as brownfield at all, because an agent that discards the given code and writes from scratch has done greenfield with a longer prompt. That form is also the one that can attach a correctness verdict to a price, because a repair is a delta against a baseline that is measurable before the agent starts. Brownfield inside a larger system, where the component under repair is one part of a codebase the agent must not break, and specification change, where the statement moves under the agent mid-task, are planned and have no registered object yet. They are named here so that the paper says what the benchmark is for and not only what it has done. The problem set has the same shape: five components in this paper, with an expansion to fourteen authored on the same form and ordered to follow the statement-arm run.

task form	`plain-bare`	`diet-bare`	`plain-statement`	`diet-statement`
greenfield	run	run	1 run, 5 registered	1 run, 5 registered
brownfield, planted defect	authored	authored	authored	authored
brownfield in a larger system	planned	planned	planned	planned
specification change	planned	planned	planned	planned

Run means cells landed and reported in this paper; *registered* means a dated amendment authorizes the cells and fixes their reading before they fire; *authored* means the task material and the form's design exist in the repository and no run is registered; *planned* means intent, with no object yet. The arm names are the registration's, and the word *diet* on the treatment records that the matrix measures a reduced rendering of the method.

S3-Systems: five components authored for this benchmark. This population is not drawn from a public set. Five classic systems components were authored for the benchmark in Rust and frozen in a dated export before any model call: `Crc32`, `FreeList`, `LRU`, `LZW` and `Paxos`. Each is frozen as four files of the same shape: a task card, a greenfield reference (`G`), a base plus a change request for a maintenance card (`B`), and the referee's name list. The frozen material runs to 9,141, 12,917, 14,643, 20,861 and 16,037 bytes across the five, and none is a stub. Each of the ten cards carries a withheld test suite, and a withheld mutant set was authored against it. An authored population has no third-party licence to reconcile and no contamination proxy to compute against a public gold patch; what it gives up is independent authorship, and Section 4 states that as a limitation rather than leaving it implicit.

The withheld suites were measured against the withheld mutants before any cell ran, by driving the same runner the referee drives rather than a second invocation of the test tool: over the ten cards, 44 of 44 mutants killed, none surviving and none unmeasured, and every reference passing its own suite. That number is a ceiling, not a strength: the mutants were authored beside the tests, and the figure is a property of the suite, not a verdict on any cell. Only `LZW`'s card carries

a `## Statement` section; the other four were authored without one, so 24 of 24 statement-arm cell builds refused at the harness, which is the harness working and a content gap in the population.

3 The protocol, as exercised on v3

3.1 The referee decides, outside the agent's elaboration

An episode ends when the agent stops or hits a cap, and what it leaves behind is scored by a checker the agent never sees, on a machine the agent never reaches. What "solved" means is fixed per substrate, and it is fixed by the referee, not by the agent.

Rust (S3-Systems), the benchmark's referee. The referee for the authored population is the withheld test suite of the card, run by the harness's own runner outside the agent's workspace, with the suite and its mutant set never shipped to the agent. That is the design. The run this paper reports on this population (Section 4) scored its cells for cost with the referee not invoked, and a declared post-hoc pass invoked it afterwards on the surviving repositories, under classes fixed before it ran (Section 4.1).

Three other referees appear in this paper. They scored the earlier reads of Section 6, the reads that shaped this design, and they are stated here because the protocol around them is the same one. The Lean referee is also the one a Lean arm of the benchmark would use.

Lean (CLEVER [16]). The checker extracts only the agent-writable bodies by section markers, screens them for command-introducing and meta keywords with comments and string literals stripped, assembles a canonical file from the frozen statements and the bodies, and compiles it under an operating-system sandbox. It then replays every declaration of the compiled module through the Lean kernel [5] (`Lean.Environment.replay`), so a declaration the elaborator admitted without the kernel is rejected. It compares, as kernel expressions, the type of every frozen declaration and the value of the human specification between the canonical file and a pristine file with `sorry` bodies, so a notation, macro, instance or `open` that changes what the frozen text means is caught by construction. It collects the axioms of the audited declarations and requires every set to be inside $\{$`propext`, `Classical.choice`, `Quot.sound`$\}$; `sorryAx`, user axioms, and `native_decide`'s `Lean.ofReduceBool` all fail here. The class names the first failing gate: `SCREEN`, `COMPILE`, `KERNEL_REJECTED`, `STATEMENT_ALTERED`, `AXIOMS_FAIL`, `PASS`.

Verus [10] (VeruSAGE-Bench [17]). Verus emits no proof object and there is no independent re-check, so the protocol says "the referee accepted" where the Lean protocol says "kernel". The integrity work that the kernel replay and axiom audit do in Lean is done here by three text and AST layers before the referee runs: a screen over the agent's regions; a count guard over `admit()`, `assume(`, `external_body` in both spellings, `assume_specification` and `verifier::admit`, canonical against original; and the benchmark's own `lynette additions` AST comparison. A screened or cheating body is never handed to the referee. `PASS` requires exit 0 and a results line that parses as `N verified, 0 errors` with $N \geq 1$ and no partial-verification suffix. Resource-limit exhaustion and compile errors are their own classes and are never charged to the arm.

SWE-bench [9] Verified [13]. The scorer is the unmodified `swebench` harness in the pinned evaluation images, with the fail-to-pass and pass-to-pass suites hidden from the agent. This substrate was abandoned after stage 0 (Section 6.2); its protocol is kept in the record.

3.2 The fence is measured, not assumed

The agent's subprocesses run under the operating system's sandbox (macOS Seatbelt through the agent harness's own sandbox settings): every host denied, writes only under the working copy and the session temp directory, reads denied on the harness state, the credential trees, and the run root. The checker runs under its own `sandbox-exec` profile. A command-text hook on the shell tool is kept as an attempt audit, not as the fence, and an audit layer records every tool read by path. Five smoke probes run before the driver starts: a canary, network escapes (including an in-language process spawn through the toolchain wrapper), reads of the arm files and the harness config, writes through the symlinked build directory, and the compile path. The driver refuses to start unless the last verdict of every probe is a pass carrying the freeze's episode-script hash.

Ground truth never reaches the host while an arm writes a specification blind. The stage-A views carry no human specification; the stage-B and stage-C views are shipped only after every stage-A episode has landed, and the ship is verified by a content set-hash rather than by the exit code of the tool that moved it.

Two findings in the record show why the fence is measured. The Verus referee's docstring claimed a sandbox, a process group and a belt; a grep of the file found only the comment, and the referee had been running unfenced. The exec set turned out to be four binaries, established by a ladder in which each arm's own error names the next binary, and a fifth exec point was revealed only by the red arm's panic text. A proof-only fixture had passed the four-binary fence, which would have been the receipt.

The third finding arrived after every read reported in this paper and applies to all of them. The sandbox's read denials bind sandboxed subprocesses: the shell tool and whatever it executes. The agent harness's own file-reading tool is executed by the harness process and never enters that sandbox. Neither substrate's settings carried a tool-permission rule, so no scored episode of this campaign had the agent's tools fenced by path. The smoke probe that had certified credential reads denied made its reads through the shell and through an in-language process spawn, both sandboxed; a probe written in the sandbox's language cannot see a hole in the layer above it, and it reads afterwards as though the agent itself were fenced. What was in force, each with its own evidence, was the subprocess sandbox, the empty network allowlist, the shell-tool hook, the ground-truth leak check, and the audit layer. That audit is what establishes the S2-Rust read: across its 15 Opus episodes no episode was voided and no unblocked read of a fenced path was recorded (one attempt through the shell was blocked). One episode of the lower-tier probe reported in Section 7 directed its file tool at a path under its own configuration directory, was not refused, and was voided as unscorable; read at the transcript the call returned that the file did not exist, so no bytes came back, and the audit field had recorded that the call was not blocked rather than that it was served. Those are different facts, and only one of them is a fence event. Over every landed S2-Lean episode in the campaign's four state roots, 278 with a transcript and 278 parsed, the count of file-tool calls at a fenced path is zero served and zero not served: no agent ever directed a file tool at a path under a deny root. The detector was driven on all three branches (a served read of a fenced path counted, a refused read classified and not counted,

a read of an unfenced path ignored), because a quiet failure reads as good news. What this measures is what the agents did, not what they could have done: the canary shows the gap was open, and an absence of exploitation is not a presence of protection. The repaired fence derives a tool-permission deny list from the same path list as the sandbox denials, 132 rules beside 11 paths, and was driven with a canary in both arms: with the tool layer absent the canary reached the agent's final message, and with it present the agent reported the file unreadable and the canary appears in neither the result nor the transcript.

3.3 Pre-registration, and the amendment discipline

The dated freeze commit is the authorization. Before it: the population rule and seed, the arms as byte-pinned files, the checker, the constants, the reading rule, the predictions, the adverse outcomes, the stop rules with their enforcer. After it: nothing above the line is edited. Every later change, including a repair to an instrument, is a dated amendment appended before its own first model call, with its own predictions. Corrections to a registered document are appended as corrections, never edited away; the record carries several such corrections by its own authors.

The reading rule on the Lean and Verus populations is a count, not a p-value: for two arms on the same drawn problems, b is the number the first arm proves and the second does not, c the reverse, and $|b - c| < 5$ is read as INDISTINGUISHABLE. The threshold is registered before the first call and the outcome table is enumerated, with the adverse cell named. For the Verus wave the fixed separation is replaced by one derived from a measured null (a same-arm rerun), because the substrate's own paper [17] reports that 11.8 % of failures flip on rerun.

Predictions are scored as they stand. Of the registered predictions in the record, several failed, and the failures are recorded as failures with their direction: the estimator over-priced five consecutive stages, then under-priced the sixth because one episode carried 64 % of a stage's spend.

3.4 A budget stop is a halt, never a failure

A per-episode token stop at four times the governing regime's p90 is registered as a halt: a halted episode is recorded as halted and is unresolved for the rate, because scoring an episode the budget stopped as a failure would let the budget instrument move the result. The multiple was chosen from the record: no passing episode among 177 had exceeded 2.40× its regime's p90, and the one runaway was at 7.79× and failed.

The rule had to be enforced where the verdict is written, not only where it is registered. The first halted Verus episode was scored `SCREEN` on a snapshot of an interrupted edit (two `assume` calls the agent had not yet discharged). The checker now takes the episode's termination and writes `class = HALT` for a token or wall stop, preserving the class it would otherwise have written as a diagnostic.

3.5 Ask of every gate which arm is more likely to trip it

The treatment arm encourages helper lemmas. Three gates in the Verus grader were found, in sequence, to refuse exactly that. The AST comparison refused a helper placed before the enclosing `impl`, classifying the episode as `STATEMENT_ALTERED`, a cheating class, for doing what the prompt invites; 47 of 207 views are impl-enclosed. The screen's rule set enforced 29 refusals

of which the prompts named 12, and the unstated `use` rule bites in the helpers region; the repair makes the prompt a consumer of the rule set, with a gate that refuses a rule with no prompt entry. The helpers whitelist then split items by brace depth and refused a legitimate lemma whose `ensures` clause carried struct literals; it was the third episode ever run that found it, after 29 screen arms, 18 fence arms and an 11-arm fixture kit had passed. An instrument that penalizes the treatment for applying the treatment does not measure a small effect badly; it manufactures the opposite one. The discipline is now a standing question at every gate.

3.6 The arms

Every arm receives the same task file and the same stage prompt; only the agent's instruction file differs, and it is byte-pinned. `a0` is the plain arm: the base block alone. `a1` is the placebo: the base block plus an equal-length process prompt with no method content (1,746 bytes against the treatment's 1,913; the base is 627). `a2` is the treatment: the base block plus the method's solo-renderable core [8]. The rendering, article by article against its source, and the articles that are not rendered because they presuppose an organisation with a human in it, were registered before the first call; no result from `a2` may be read as a test of the method's multi-agent tier. The treatment text names `sorry`, which is the control's dominant failure; the objection that the arm coaches to the metric was registered in advance and is answered by the data in Section 6.3.

4 The v3 reading: a registered cost premium on all five problems, and a post-hoc correctness pass that did not separate the arms

This is the only reading in the paper that returns a difference between the arms, and the difference is in price, not in what a referee accepted. The registered reading is a cost result. No cell of the run carried a correctness verdict at the time it was scored, and the paragraph on correctness below states and measures that fact; Section 4.1 reports the declared post-hoc pass taken afterwards. That is a statement about what was run and not about what exists: the withheld suites were authored with the population and characterised before the first cell (Section 2), and the cells were scored for cost with the referee never invoked on them.

The design, registered before the first cell. Five problems, four arms, $n = 3$ per condition, at `claude-opus-5` on the greenfield card. The arms are `plain-bare`, `diet-bare`, `plain-statement` and `diet-statement`. The treatment arm is a *dieted* rendering of the method [8]: prescribed acts were removed after the full rendering hit the per-cell cost cap on a single earlier cell at $42.24, and the registration requires the word *diet* on every headline, because this matrix does not measure the un-dieted method. The primary reading is a cross-problem sign test on $\text{premium}(P) = \text{median}(\text{diet-bare}, P)/\text{median}(\text{plain-bare}, P)$, one premium per problem, against the null that each premium is equally likely either side of 1.0. The outcome table was enumerated before the first cell: 5 of 5 gives $p = 0.0312$, 4 of 5 gives $p = 0.1875$ and is registered in advance as *not* a positive result, 3 of 5 gives $p = 0.5000$. None of those numbers may be quoted on its own: the four qualifiers stated with the reading below were registered with the test and are part of it. Five problems is the smallest count at which a clean sweep can reach .05 at all, which is why the earlier two-problem stage could not have produced a verdict whatever its cells had said.

The client was pinned by absolute versioned path to the build the earlier stage had used, rather than resolved from the shell path, and the registration names the cost of that choice: the matrix measures a client two releases behind the one a path lookup would have fired. A symbolic link follows the newest install, so resolving a client from the path silently re-pins a scored run every time the vendor ships.

What was built, against what was registered. The registration and the campaign price cover 60 cells; 36 of them were buildable. (*Priced* is used in its own sense in the census below, where it means a cell that produced a cost, and the two senses are kept apart deliberately.) The 24 statement-arm cells on the four problems whose cards carry no `## Statement` section refused at the harness (Section 2), which leaves the bare pair complete on all five problems and the statement pair complete on `LZW` alone. The condition key is `(task, arm, card_extras)` read from each cell's own control directory, and a scorer keying on the arm alone would have pooled the bare and statement arms of the same treatment, doubled each apparent n, and mixed the very pair the design calls its gold, with no error raised and medians that look reasonable. The scorer that had scored every earlier stage correctly keys on two fields, because until this matrix a third was never needed.

The scoreboard, at two readings. The declared set is scored twice and the instrument computes both readings itself, so the comparison below is the tool's output and not a patch applied to it. Reading A is the continuity reading: the matrix root, the three cells AMENDMENT 26 added, and the three borrowed smoke cells, 43 cells in all. Reading B keeps only what this run produced, dropping the three borrowed cells, 40 cells. Both exist because of a dependency the first scoreboard did not show, which is stated below the table.

problem	premium, reading A	premium, reading B
Crc32	1.1610×	1.1610×
FreeList	2.8070×	2.8879×
LZW	1.3749×	1.3749×
Paxos	2.4306×	2.2437×
LRU	1.2826×	1.1521×
sign test	5 of 5, $p = 0.0312$	5 of 5, $p = 0.0312$

Cost is US dollars of metered subscription spend per cell, read from each cell's harvested `METER.txt`. Every premium is a ratio of medians at $n = 3$. An earlier version of this table carried the two medians behind each ratio; they are not printed here, because the instrument that computes these two readings prints the per-cell prices and the premium and does not print the median, and a median recovered by hand from a printed cell list is a number this paper cannot cite to a file. The per-cell prices for every condition under both readings are in the result file. Three of the five premiums sit below the design's resolvable floor of 2.0072× under both readings. The columns are printed so that the reader can reconstruct the sign, and by the registration no entry in either may be read as the finding.

The reading, and the four qualifiers that are part of it. Five of five premiums exceed 1 under both readings. On the registered one-sided sign test that is $p = 0.0312$ at each. The four qualifiers

below were registered before the first cell fired, and the campaign's own rule is that they travel with the p-value wherever it is quoted; a reader who has the number without them has a different claim from the one this design supports.

1. **The test is one-sided.** It can confirm the hypothesis it tests and cannot significantly refute it: five premiums *below* 1 would have returned $p = 1.0000$. A one-sided test is a statement about which surprise the design was willing to be surprised by, and this one measured the expected surprise.
2. **No magnitude is resolvable for the population.** At $n = 3$ with the registered pooled $\mathrm{sd}(\ln \mathrm{cost})$ of 0.30458 the smallest resolvable premium is 2.0072×. Three of the five fall below it under both readings: Crc32 at 1.1610, LZW at 1.3749, and LRU at 1.2826 under reading A and 1.1521 under reading B. Only FreeList and Paxos clear it, and two resolvable magnitudes out of five are not a magnitude for the population. No ratio may be reported as the finding: not 2.7×, not a range, not a percentage. The finding is a sign across five problems.
3. **The paired arm the design calls its gold reaches no verdict.** It exists on `LZW` only, which is $k = 1$ and $p = 0.5000$ at any outcome. Its cells are plain-with-statement at \$10.24, \$11.39 and \$9.82 against diet-with-statement at \$22.53, \$15.34 and \$18.54, a ratio of medians of 1.8105×, and it carries no verdict.
4. **The within-condition noise is larger than the smallest premium.** `FreeList` on the plain arm spans \$13.01 to \$23.38 inside one condition on one problem, a spread of 1.80×, against the smallest premium in the table, which is 1.16× on Crc32 under reading A and 1.15× on LRU under reading B. The registered spread clause says why the count of wide conditions is not itself a reading: at $n = 3$ with this dispersion a perfectly homogeneous condition crosses a 1.5× spread with probability 0.614, so about twelve of the matrix's twenty conditions are expected to look wide by sampling alone.

What may be quoted from this reading. The finding is the sign: the dieted treatment cost more than the plain arm on every one of the five problems, one-sided $p = 0.0312$, at $n = 3$ per condition. The per-problem premiums in the table above are observations. They are printed so that a reader can reconstruct the sign and see the spread it was taken over, and the registration does not license any of them as a result. A span taken across the five, such as the interval from the smallest to the largest, is a range of three-cell medians and not an interval estimate for a population: this design registers no population magnitude, three of the five premiums fall below its own resolvable floor under both readings, and the within-condition spread on one arm of one problem is larger than the smallest premium. A summary of this result may therefore say that the treatment cost more on all five problems, and may quote any premium as an observation on its own problem at its own n. It may not state a ratio, a range or a percentage as the cost of the method, and it may not describe the result as a measured cost multiplier: not 2.7×, not one to three times, not a percentage, and not an interval of any width. Those are claims about a population this run did not measure, and an interval narrower than the within-arm spread reports a precision this instrument does not have. The three premiums below the floor are the ones such an interval would carry, and for those three the design cannot say that the true value is above 1.0 at all. No cell of this matrix carried a referee verdict on a withheld suite when it was scored, so the cost reading bears on nothing about whether the method changes what a referee accepts; the post-hoc pass below did not separate the arms on that question, and it remains the question the next wave exists to answer. A reader should not take that absence for an absence of

tests: the suites exist, and the sentence that is true of this run is that the referee was not pointed at its outputs when the cells were scored. Section 4.1 reports a declared post-hoc pass over the surviving repositories; a post-hoc suite outcome is still not a pre-registered capability result, and that pass did not separate the arms.

4.1 A post-hoc correctness pass

This pass is post hoc. Matrix 1 was registered as a cost experiment, its cost result was known before any suite was run, and the classes below and their meanings were fixed in a dated pre-specification before the first cell was scored. It is reported as a declared post-hoc measurement and not as a pre-registered result. The referee is the same runner the withheld suites were characterised with, it is arm-blind by construction, and the pass cost no model tokens: 43 cells in 246 seconds.

class	cells	class	cells
PASS	35	CAP-COST	3
FAIL	1	FAILED-BOOT	4
NO-BUILD	0	INTERFACE-MISS	0

Every class is printed, including at zero, because a dropped cell is a claim that it did not exist. CAP-COST and FAILED-BOOT are not failures of the method: a cell stopped by the budget, or one that produced no subject behaviour, was never asked the question, and neither is pooled into FAIL. The distinction is not decorative, because applying it changed the answer. The runner returns non-zero for a cell that never booted, so a first tally read five failures, and the frozen classes reduced that to one. Four cells that produced no subject behaviour would otherwise have been published as correctness failures.

The registered reading: the instrument did not separate the arms. Landed cells on the bare arms, per problem, as cells passing over cells landed:

problem	plain	salt-diet
Crc32	3/3	3/3
FreeList	4/4	0/1
LRU	4/4	3/3
LZW	3/3	3/3
Paxos	4/4	2/2

Four of the five problems tie, and one pair is informative. The cross-problem sign test cannot reach significance on this and does not try. That outcome was registered in the pre-specification before any suite ran: that pass counts may be equal on every problem is written there as a registered outcome, named as the instrument not separating the arms, and it is stated not to be a failure, not a null to be spun, and not grounds for a second analysis chosen afterwards. There is no second analysis. The single difference is one cell failing one test of seven, and that is not an adverse finding and is not reported as one. Reported alone, the `FreeList` row misleads: of the treatment's four cells on that problem, two were stopped by the budget and pass 7 of 7 when their suites are run, one never booted, and one landed at 6 of 7.

Being stopped by the budget is correlated with the arm. The classes are not distributed evenly across the arms. On the bare arms the control contributed 18 landed cells and none stopped by the budget, and the treatment contributed 12 landed cells and three stopped. Every budget-capped cell in this matrix is a treatment cell. The correctness column therefore scores 18 control cells against 12 treatment cells on those arms, and the treatment cells missing from it are the ones that ran long enough to reach a cap, which is to say the expensive ones. The surviving treatment sample is easier than the arm it is drawn from, so any pass rate computed for the treatment on this run is biased upward by construction. This is a selection effect, it is correlated with the arm under test, and it travels with every correctness number from this run. Section 5 states it as a property of the instrument rather than of this population.

A budget stop is not a correctness failure, and on this run that is measured rather than assumed: all three capped treatment cells pass their suites completely when they are run, 7 of 7 and 7 of 7 on `FreeList` and 17 of 17 on `Paxos`. That is why the pre-specification gave CAP-COST its own class instead of letting it read as a loss.

What a pass is, and what this section does not license. A pass is that the withheld suite did not fail the cell. It is not a proof of correctness. The suites were characterised against authored mutants at a ceiling rather than a floor (Section 2), and a suite that kills every mutant written beside it has been shown not to be vacuous, not to be hard. Nothing in this section licenses joining the cost result to this one. In particular it does not license the claim that the treatment costs more and produces more correct code, which is a causal join this design cannot support, and it does not license a correctness comparison between the arms, which the reading above says the instrument did not make.

The declared set, and a dependency the scoreboard does not show. The scored set is declared by identifier rather than by a glob over the archive, because cells from different runs share the condition key and a glob pools them: an early draft of the scorer, driven on partial data, silently pooled this matrix with the earlier stage, with a pricing set, with two dropped arms and with a void cell. The declaration is the cells of the matrix root plus three named smoke cells, `ae304f63`, `a69e9131` and `b7537006`, one each on `FreeList`, `LRU` and `Paxos`, fired first as an end-to-end smoke on the three problems that had never produced a cell and registered in advance as counting toward their conditions. To that declaration AMENDMENT 26 added three further plain-arm cells, one each on the same three problems, fired in a third cells root of their own with a fresh identifier prefix and against a subject-facing tree driven byte-identical to the matrix's before the first of them started. All three landed, none hit the cost cap, none was void, and each was priced twice by instruments that agree to the cent: the harvested `METER.txt` and the cell's own end-of-run control record.

Those three cells live in a different cells root from the rest of the declared set, and the matrix root holds two landed plain-arm cells each on `FreeList`, `LRU` and `Paxos`. Three of the five problems therefore reach $n = 3$ on the plain arm only by counting one smoke cell each, and without them the registered rule takes no median on those three: the sign test becomes 2 of 2 at $p = 0.25$, which is no verdict. This is not a defect in the declaration, which names the three by identifier and says why a glob would be wrong. It was a fact about the dataset that the scoreboard was silent about, and it travelled with the number until the cells that retire it landed. The three smoke cells also ran under an earlier export of the harness whose repairs are to the

audit, carrier and canary paths and not to the task, the arm, the prompt or the caps; on that basis they contribute a price, and their containment verdicts are withdrawn rather than caveated, so no containment claim for this matrix rests on them.

The dependency is discharged for the sign, and not for the magnitudes. The rule for reading that discharge was registered with the top-up and before its cells fired: report the sign test with and without the borrowed cells, and if the two readings agree the dependency is discharged, while if they diverge the divergence is the result and is reported ahead of the headline. Both halves fire at once here, and reading only the first is the error this paragraph exists to prevent. The readings agree on what the registration calls the reading: 5 of 5 at $p = 0.0312$ under A and under B alike, so the headline does not rest on the three borrowed cells and the dependency reported above is retired. The magnitudes do not agree. Three of the five premiums move between the readings and two of them move down: LRU from 1.2826× to 1.1521× and Paxos from 2.4306× to 2.2437×, both because the cheapest plain cell on each problem is a borrowed one and its departure raises the plain median, and FreeList up from 2.8070× to 2.8879× because the cell that leaves there sat near the middle. Crc32 and LZW do not move, having no borrowed cell to lose. Reading B's LRU premium is the closest to 1.0 that any premium in this campaign has come, and the borrowed cells were flattering it. No qualifier is retired by any of this: three of the five magnitudes remain below the resolvable floor under both readings, so the movement of a magnitude is barred from being the finding in exactly the way the magnitude itself is. That is the reason it would be easy to call the movement immaterial, and it is not immaterial. It is the measurement of how much the borrowed cells were flattering the table, and a dependency can be discharged while the thing it was propping up gets weaker.

What the top-up did not do. Before its cells fired, the amendment computed the premium range each problem could reach at $n = 4$ and showed that no outcome of the top-up could move any premium to 1. All three landed values fell at an end of their registered range, which is what a fourth observation does to a median taken as the mean of a middle pair. Because the exercise could not have changed the verdict, it did not confirm it: it bought precision and removed a dependency, and it is not a replication of the earlier reading and is not written as one.

The declared set spans two run accounts, which is the third thing the scoreboard did not show. The three cells the amendment added wrote their transcripts under one run account and every cell of the matrix root under another. Neither result file showed this: one named its own account and not the contrast, the other names no account at all, so a reader of either could not tell that a set described as every cell from this run pools cells recorded under two. It does not move a price, and that is measured rather than assumed, because both populations are priced from the same rate card read on the same day and a cost here is a rate card applied to a token count rather than an account's bill. What is not measured is the one mechanism that could bite. An account cannot change the price of a token but it can change the count, since almost all of the token total on these cells is cache read and cache state is held per account and per session, and nothing here measured whether that differs systematically across the boundary rather than randomly. Cache state already varies from cell to cell inside the matrix, so this is not a new source of variance; it is a possible systematic one and it is unquantified. Every cell from this run is a claim about a run, and a run is not necessarily one context.

Correctness: no cell of this run carried a referee verdict when it was scored. Of the cells in the matrix root, 33 are priced, and the three cells AMENDMENT 26 added take reading B's declared set to 36 priced. None of them carried a referee verdict on a withheld suite at the time it was scored, and the three added cells added none: no quantity of cost cells closes a correctness gap. The path such a verdict would be read from did not exist then: five patterns swept across the whole cells root return zero files each, and the scorer reads a cost off the archive and no verdict of any kind. The only per-cell artifact that resembles a check is a manifest integrity check, which is not a correctness verdict. The withheld-suite work that does exist characterises the suites against mutants (Section 2) and is a property of the suites, not of any cell. This does not say the code was wrong. It says no instrument in this run asked when the cells were priced, so the premium is a price for work whose correctness the run did not verify as it went, and no sentence in this paper pairs it with quality or with working code. The post-hoc pass of Section 4.1, run afterwards against the surviving repositories, is the instrument that asked; it did not separate the arms, and it does not change what the premium is a price for.

Two counts in that measurement do not reconcile and are reported rather than smoothed. Priced does not imply landed: of the 37 cells with a control directory in the matrix root, 30 landed, 3 stopped at the cost cap and are priced without having finished, and 4 failed to boot and are unpriced, which gives 33 priced. An earlier hand census in the same file reports 36 cells, 32 priced and 4 void. The file states the difference instead of adopting one number, and neither count changes the correctness answer, which is zero. Pricing a capped cell beside a landed one prices two different events under one name, and the three capped cells are named here for that reason.

The placebo arm supports nothing. An equal-cost placebo arm was run to completion on all five problems: 15 of 15 cells landed and priced, $193.42, no void cell, no cap, no drift and no failed boot. It returns UNRESOLVED on all five problems, every one inside the band $[0.4982, 2.0072]$. That outcome was the predicted one and was registered as such before any placebo cell existed, because the floor at this n is $2.0072\times$ while the treatment's own premiums on three of the five problems sit below it. The sentence this result most invites, that the placebo came in near the plain arm so the treatment's premium is method rather than form, is forbidden by name in the registration and is not written here. An arm that could have damaged the finding and did not is worth its price and is still not evidence for the finding.

A cross-stage cost claim is confounded by concurrency. The earlier two-problem stage ran mostly one cell at a time, read from its own launch receipts; this matrix ran four-wide in both its fire and its resume schedulers. The two sets of premiums therefore differ in box contention as well as in date, and nothing in the record separates those. The confound does not touch this matrix's own result, which is computed entirely within a run where both arms were measured four-wide; it bars reading the matrix's premiums as a replication of the earlier stage's magnitudes, and the placebo was fired at four-wide for the same reason. This correction was made to the campaign's own earlier claim: the two stages had been established to share a client binary, byte for byte, and were then treated as cost-comparable on that basis, which verifies one axis and generalises the verdict past it.

Every cost in this section carries an unmeasured box-load term. The concurrency confound above is a known difference between two stages. The wider one is that nothing in this campaign measures what the box was carrying while a cell was priced. The harness reaps the client and the watcher and does not reap what the subject forked: 24 processes forked by the subject of one cell in a later wave were re-parented to the init process and ran for three hours and twenty-three minutes, outliving their own cell's end by three hours and nine minutes, with 11 of the 16 priced cells of that wave metered wholly or partly inside the window. A cell's end is not the end of the cell's processes, so any cell in this campaign may have been priced on a box carrying the residue of earlier cells, and no arm looks, so no run can state its own load. The direction of that on cost is unmeasured and is not asserted here in either direction: extra processor time does not spend tokens, and a route from load to price through timeouts or extra turns was never driven. For this matrix specifically the term is bounded rather than assumed: all 37 cells of the matrix root reached their end at or before `2026-09-09T00:51:59Z` and that leak opened at `2026-09-09T03:34:51Z`, a margin of two hours, forty-two minutes and fifty-two seconds, so no cell of the matrix root was metered inside it. That bound is stated over the matrix root, and the declared set is now larger than the matrix root: the three cells AMENDMENT 26 added ran later on the same day, and no file in this repository records an end time for them, so the margin above does not cover them and this paper does not claim that it does. The bound therefore clears the matrix root of that leak, and of no other, and leaves three cells of the declared set unbounded for it. A set that grows after a bound is written over it does not inherit the bound, and the growth is silent: nothing in the scoreboard, the premium or the p-value changes shape when a cell that no timing record covers joins the set.

What this population cannot do, stated with the result. It is authored by the same people who wrote the treatment, so it carries no independent authorship and no contamination measurement of the kind a public set allows. It measures a dieted rendering of the method and not the method. It measures price and not acceptance. Three of its five magnitudes are below its own resolvable floor and none of the five may be reported as the finding. Its gold pair is $k = 1$. And three of its five problems currently reach $n = 3$ on the plain arm by counting a cell from another root. Version 2's remedy for the last of these is under way and is described in Section 7.

5 The instrument findings, as results

These are the results we think transfer. Each was found by a gate, a probe, or a re-check in the record; each changed the protocol; and each is stated as the rule it taught, with the episode that taught it beside it.

1. **A censored cap returns the cap.** Section 6.5. Any budget derived from a capped distribution has to come from an uncensored read, or from a cap raised until the censoring fraction is near zero. Recording the state each episode had reached when the cap cut it does not rescue the estimate on its own, because under a verifier that state can carry no information: an obligation is discharged or it is not, so an incomplete proof reports zero discharged until the moment it reports all of them. A partial count is a distance and a zero count is an absent measurement, and Section 7 reports an episode that read zero at the cap and held a complete verified proof thirty-nine calls later.

2. **A statistic is a cap's price only in the cap's unit.** The quota cost per token at the higher tier was roughly double, while the token count fell; a campaign that priced the tier on tokens alone would have called it free. The metered sum and the quota unit are different quantities and a stop rule has to say which it uses.
3. **A gate can penalize the treatment for applying the treatment.** Section 3.5, three instances in one wave, the third inside the layer added to prevent the first.
4. **A screen's false positives are invisible by construction.** The campaign ran 201 scored Lean episodes and examined exactly one refusal, because exactly one existed; it was a complete proof. A defence-in-depth layer needs its own false-positive reading routinely.
5. **A blind agent that passes is a measurement of a different task.** A fence derived from the run root denied the agent its own episode directory, so the referee wrapper was unreachable; the agent wrote proofs it could never check, and two of them passed the referee at 11 to 16 times the cost of a sighted episode. The episode driver now refuses when the fence and the workspace intersect.
6. **A deny list is only as broad as the layer that enforces it.** Section 3.2. The sandbox's path denials fenced the agent's subprocesses and not the agent's own file tool, and the probe that certified the fence made its reads through the sandbox, so it could not see the layer above. A fence has to be probed in the language of every tool it is meant to bind, and the probe has to be neutral: a probe that told the agent a tool was denied and asked it to route around was answered by the agent's judgement with no tool call made, and read as blocked while measuring nothing. The audit built to find such a hole in the record had the same shape of defect: its field recorded that a call was *not blocked*, which conflates a denial, an absent file and a served read, and it reported an episode as an escape whose read had returned that the file did not exist. Only a served read is a hole being used, and a field that cannot say which of the three it saw will report the wrong one.
7. **An instrument that states a reporting rule in the grammar of a measurement will be quoted as one.** The scorer for the systems population printed its floor rule as the sentence *every per-problem magnitude is unresolved*. That is how the rule reads when it is taken as a statement about the table rather than about what may be reported from it, and the abstract of this paper quoted it faithfully and was false until it was corrected: the same run's table prints two premiums above the floor and the body of the section says which two. The defect was in neither the number nor the prose but in the instrument's wording, and no gate could fire on either half, because reading the paper against its own table finds the symptom and only reading the scorer finds the cause. The scorer now names the two sets apart and says in its own output that the rule is not a claim that every magnitude fell below.
8. **An absence is only as wide as the population the instrument could see.** This paper reports absences as results: five patterns swept across a cells root returning zero files each, and zero file-tool calls at a fenced path across 278 landed episodes. Each was measured with a positive control, which is the right discipline and is not sufficient on its own. A sweep of this repository's own references for a registered amendment returned absent with its positive control firing correctly, and the document existed throughout on a remote the working copy had not fetched, so the control was drawn from the same truncated population as the search and could not have detected the truncation. A positive control establishes that the instrument works on the population it can see. It says nothing about whether that population is the one the claim is about, and an absence sweep should therefore name the population it enumerated.

9. **A bound does not cover a set that grew after it was written, and the growth is silent.** Section 4. The box-load term for the systems matrix was bounded by showing that every cell of its root ended before a process leak opened. Three cells were later added to the declared set by amendment, from a different root and a later hour, and no file records an end time for them. Nothing in a scoreboard changes shape when a cell that no timing record covers joins the set: the premium is still a ratio of medians and the p-value is still a sign test, so a bound stated over the old population reads as though it covered the new one. A bound has to name the set it was taken over, and a declared set that changes has to re-derive every bound stated over it.
10. **A gate that extracts by delimiter measures the delimiter.** A statement-immutability failure was reported as the agent altering the specification; the specification was byte-identical, and the mechanism was Lean naming a cached auxiliary `match` declaration after whichever declaration elaborated it first. The audit now compares matchers by content.
11. **A guard whose predicate cannot hold looks like a guard in every review.** The recall instrument's provenance guard compared a wrapper's hash to its wrappee's and had taken the fallback on 78 of 78 rows; the fallback read the same bytes, so no number moved. The token ceiling had been blind on every episode where it could have mattered, because the watchdog's call crashed on the first path-carrying tool call and the shell turned the crash into zero; repaired with a self-test that makes the caller's exact call.
12. **A scoreboard is silent about the structure of its own dataset.** Section 4. Two properties of the cost matrix are invisible in the number it produces: no cell in it carries a correctness verdict, and three of its five problems reach their registered n on the plain arm only by counting a cell fired in a different run and a different cells root. Neither is a defect in the arithmetic, and neither can be seen from the arithmetic. A result that is reported without them is a different claim from the one the data supports, so a scoreboard needs a companion statement of what its own population is made of.
13. **An instrument can be right on everything it has ever scored and wrong on the next design.** The condition key for the earlier two-arm stages is the problem and the arm. The matrix adds a third arm dimension, and a scorer keying on two fields pools the bare and statement arms of the same treatment, doubles each apparent n, and mixes the pair the design calls its gold, raising no error and producing medians that look reasonable. It was caught by building one cell and reading its control directory before building the other 56.
14. **A control arm that could only have hurt you is worth its price and is not evidence for you.** Section 4. The placebo arm on the systems population cost $193.42 and returned UNRESOLVED on all five problems, which was the outcome its own registration predicted from the floor at that n. It could have cleared the floor and damaged the finding; it did not. The reading that the placebo therefore supports the treatment was written down as forbidden before the arm was fired, and the question of what an arm can resolve is free before the cells and is priced at the arm's full cost afterwards.
15. **A counterfactual about someone else's instrument must be run against their code.** Section 6.4: the headline that the reference checker was sorry-inflated was refuted by its own kill-check, and what survived was a different hole.
16. **A stop rule that is correlated with the arm removes the treatment's hardest work from every sample taken later.** Section 4.1. The cost matrix carried a per-cell budget cap. Every cell it stopped was a treatment cell and none was a control cell, which is not a coincidence: the

treatment is the arm that spends more, so it is the arm that reaches a spend cap, and it reaches it on the problems that take the longest. When a correctness pass was run over the surviving repositories afterwards, the cap had already decided which cells could be in it. The treatment entered that pass with fewer cells than the control and with its slowest cells missing, so any pass rate computed for it is biased upward by construction, and the bias is invisible in the pass rate itself. The cap was registered, published and reported throughout as a cost stop; nothing about it was hidden. What was not foreseen is that a stop rule silently defines the population of every study built on the same cells afterwards, and that when the stop is arm-correlated the resulting sample is not merely smaller but easier on exactly one arm. A cap has to be read as a sampling instrument and not only as a spending one, and any downstream analysis over capped cells owes a statement of which arm lost cells and which cells they were.

6 The reads that shaped the design

Three populations were measured before the seat-as-subject matrix. Each of them closed a route that had looked open, and together they are the reason the design of Section 2 is shaped as it is: a referee that decides without the agent's testimony, an authored population, and a cost question asked where a capability question had saturated. They are reported here compressed to that role.

6.1 Provenance of the three prior populations

S2-Lean: CLEVER Task 1. `trishullab/clever` [16] at commit `8348039`, MIT, Lean `v4.27.0` [5], mathlib `a3a10db0` [11]; 161 problems. A raw problem file is re-cut by the harness into three views: stage A (write `generated_spec` from the docstring, ground truth absent), stage B (prove `spec_isomorphism` between the agent's frozen specification and the revealed human one), stage C (implement and prove `correctness` against the human specification, tests included). The draw sorts the real ids by a committed seed after removing the four structural exclusions of the benchmark's agentic-proving paper [14]. Stage 0 ran $k = 30$. The registered comparison population is U15, the first fifteen unflagged ids of that order (the same paper flags 80 of 161 specifications), reached at depth 27. Stage C's population removes the two U15 ids whose stage-C view fails to elaborate before any agent text (problems 54 and 112) and one further id, problem 18, whose task is machine-checked unsatisfiable (Section 6.4), leaving 12.

S2-Rust: VeruSAGE-Bench proof targets. `microsoft/verus-proof-synthesis` [17] at commit `cbf9c0c6`, MIT; the Anvil-Advanced [15] and NRKernel [10] projects, 267 records. The wave takes the unique `proof fn` target whose body is empty modulo whitespace: 207 records; the five refuse classes are counted and published with the draw. Stage 0 then ran every reference proof through the pinned referee inside the harness's own scaffold: 180 live, 27 with a task text that omits definitions its own ground truth supplies, zero dead tasks, zero pin defects, zero scaffold defects. The pin itself was measured: the current Verus release failed 5 of 15 reference proofs at the Rust front end, and the benchmark's own release passed 15 of 15; a front-end error on a reference file indicts the toolchain, never the task. The resource-limit curve is flat into the registered limit (179 of 180 at 10, 180 of 180 at 50 and at 250) and three repeats at the pinned seed produced zero flips.

S1: SWE-bench Verified. `princeton-nlp/SWE-bench_Verified` [13] at revision `c104f840`, 500 rows, content-pinned by a hash over the canonicalised rows. The selection script is the normative form of the criteria; the draw is measured-50 and pilot-30 with a per-repository cap of 9, chosen from arithmetic over the eligible counts after the first draft's cap starved the draw in silence.

No task text from SWE-bench Verified is redistributed. Its dataset card carries no licence field, so rather than rely on a reading of what the issue text permits, the repository ships the 30 identifiers, the pinned revision, a hash over the 500 canonicalised rows and a hash over the 30-row projection the episodes read; `harness/fetch_problem_statements.py` rebuilds that projection from the pinned revision and verifies it against both. The rebuild was compared byte for byte against the file the episodes read before that file was removed from the tree.

Licences and attributions for every population in this paper, the authored one included, and the exact list of what this repository redistributes from each, are in `PROVENANCE.md`.

6.2 SWE-bench Verified saturates at Sonnet 5 and is abandoned as a substrate

Stage 0 ran the two control arms on the first 15 tasks of the pilot draw at `claude-sonnet-5`, effort high, 40-call cap, in the pinned images; pre-flight and gold controls passed 15 of 15 each and nothing was excluded. Both arms resolved the same 13 and failed the same 2, so $b = c = 0$, and the two failures are the two tasks on which both arms hit the cap, which makes 13 of 15 a lower bound at that cap. The pre-registered contamination proxy, cut at 0.80 and stated before computing, read 6 of 13 resolved tasks as high-similarity, which the rule classifies as INDETERMINATE; a blind panel found, and the record verifies at the transcript, that on two unresolved tasks the agent wrote lines of the upstream fix as its own edit input before any read could have shown them. The substrate is saturated at this tier for this scaffold, contamination is present and unquantifiable, and the treatment arms were never run on it. It is the reason the campaign moved to referees that decide rather than to suites an agent may have seen.

6.3 S2-Lean: a pre-registered null, and a tier that moves both arms

On the registered U15 at a uniform 100-call cap, `claude-sonnet-5`, stage A then stage B with arms alternating per problem, every arm proved 8 of 15. The treatment against the plain arm read $b = c = 0$: the treatment matched the control's set on all fifteen problems, and seven of its episodes ended with `sorryAx` in `spec_isomorphism`, the identical failure on the identical set. Totals were flat at 50.7M, 51.7M and 51.4M metered tokens. The paired split was not: on the eight problems both arms proved, the treatment used 15.79M tokens against the plain arm's 20.76M ($-24\,\%$), and on the seven both failed, 35.62M against 29.92M ($+19\,\%$). At $n = 8$ and $n = 7$ this is reported as a paired observation and not as an effect.

Raising the tier to `claude-opus-5` on the same U15 and the same checker moved both content arms to 10 of 15 with $|b - c| = 0$, at 26.1M tokens against the Sonnet run's 108.9M, and the raise is not monotone per problem. A registered differential re-check of one treatment episode refused by the pragma screen found a complete kernel-checked proof with and without the refused line, which reads the treatment at 11 of 15 as a diagnostic and leaves the contrast INDISTINGUISHABLE. The placebo at Opus proved 9 of 15, a strict subset of the plain arm's ten, landing on the boundary of its registered arm-independence band $[9, 11]$ by one problem's margin; the tier's gain is a property of the tier and not of prompt content. On the registered twelve stage-C problems at

Opus the plain arm certified 12 of 12, at which the reachability bound $c \leq n - P_0$ is $c \leq 0$, so no problem remained for a treatment arm to win and it did not run.

6.4 The triage: most of the null belongs to the oracle

A blind triage of every failed stage-B cell in the record labelled each from the two specification texts and the proof alone, with tier, arm, class and termination withheld. Of 31 cells, 30 are triageable: 17 are HUMAN-LOOSE, the agent's specification strictly stronger than the human one with a kernel-checked witness of non-isomorphism recorded where one could be written, 5 are AGENT-WRONG, and 8 are PROOF-HARD. The label is a property of the problem rather than of the arm: eight failing problems carry eight single labels with no exceptions, so the arms fail for the same reason on the same problems. Under four of the five false obligations the pattern is one defect: the human specification guards its conclusion with a well-formedness hypothesis and says nothing outside it, while the agent's, asked for as a `Prop` over the same signature, is total, and a guarded specification and a total one are never isomorphic.

Problem 18 is worse than loose. Under Lean `v4.27.0` slice equality makes the reference specification's occurrence test false at a genuine occurrence, so the specification forces the answer 0 where the benchmark's own test demands 3; that no implementation satisfies both is machine-checked with axioms exactly the allowlist and no `sorry`. The elaboration check that guards the population had marked it fine, because it measures whether the task can be stated and not whether it can be done. An audit of the benchmark's own reference checker adds that it agrees with ours exactly on non-adversarial artifacts and not at all on adversarial ones: its submission path recompiles the solver's view, so replacing the theorem with `True` certifies 15 of 15 without helper lemmas, and an axiom discharging the real goal certifies 15 of 15 because an axiom is not a `sorry`.

6.5 S2-Rust: the plain arm saturates the hard band at Opus 5

The upper tercile of the 180 live tasks by reference-proof wall was registered as the hard band, with its 13 drawn ids, before any of it was spent on; the cutpoints are ranks because absolute walls re-time 2.2 to 2.8 times faster on an idle machine while the order is stable. With a registered sequential early stop, 10 of the 13 were run: 10 scorable, zero void, and 9 of 10 passed at a 40-call cap, the stop firing at 10 because the ceiling threshold of 9 was already reached. The registered prediction was 5 to 9 with a point estimate of 7, so the band held at its top edge and the point estimate was low by 2, the fifth consecutive under-estimate of the plain arm.

The cap sits in the shoulder, and this is what the number costs. Uncensored passing call counts were 23, 26, 27, 27, 32, 35, 35 and 39 against a cap of 40; two of ten episodes were at the cap, one of them a pass at exactly 40, and the single failure died at 40 calls reading eight verified and one error. A p90 computed from data the cap produced returns the cap, so the measured 9 is a lower bound on the ceiling and the room left for a treatment arm is smaller than the count shows. That a cap of this size can cut an episode which would otherwise have passed is evidenced directly rather than inferred: at the lower tier on this same band, an episode reporting no discharged obligations at the 40-call cap carried a complete verified proof at call 79 once the cap was raised. The median episode spends 81 % of its tokens before its first clean referee run, range 64 to 91, so cost is dominated by search rather than verification and a cap cuts search rather than polish.

7 The treatment question, open

Nothing in this record shows an effect of the treatment on what a referee accepts. On S2-Lean the treatment arm matched the control's set at Sonnet and was within one problem of it at Opus, with the placebo moving with the tier; the triage attributes most of the shared failures to the reference oracle. On S2-Rust and on stage C the plain arm is at or near the ceiling of every band the campaign can afford, so a comparison has no room. On S3-Systems the treatment arm cost more on all five problems under the registered sign test, and that is a difference in price carrying the four qualifiers of Section 4, on a run in which no cell was checked for correctness as it went; the post-hoc pass of Section 4.1 checked the surviving cells afterwards and did not separate the arms. A price difference is not a capability difference, and nothing in that design licenses reading one from the other. The question is open. The tests that could close it are these, and with the task forms of Section 2.1 they are the benchmark's roadmap:

1. **A lower tier on the hard band.** Registered as both arms at `claude-sonnet-5` on the same 13 Verus ids with the same early stop, quoted from a Sonnet probe on the band rather than from the Opus figures. The quote landed; the read was withdrawn; and the reason it was withdrawn is the result reported here. The probe paired three of the band's tasks across the two tiers, the same task and arm and instrument with only the model differing, at the same 40-call cap. Its aggregate paired token ratio was 1.55 (per-episode median 2.06), which prices the lower tier at 0.71 of the higher tier's per-episode quota and passes the affordability gate. But Sonnet passed 0 of the 3 where Opus passed 2, and all three Sonnet episodes ended at the cap, so a 13-id read at that cap would have been in part a measurement of the cap. One further episode was run to separate the two: the same task, the same tier and the same arm, at a cap of 120 calls instead of 40.

model	cap	calls	metered tokens	wall (s)	referee at the end
`claude-opus-5`	40	23	957,722	307	9 verified, 0 errors (pass)
`claude-sonnet-5`	40	40	3,051,144	768	0 verified, 1 errors (stopped at the cap)
`claude-sonnet-5`	120	79	8,375,623	1071	10 verified, 0 errors (pass)

   The 120-call episode passed cleanly: no screen violation, no helper-shape violation, the linter at return code zero, the count guard unchanged, the resource limit inside budget, the episode not void and no unblocked read of a fenced path. The cap was therefore binding at this tier on this task, and the identical Sonnet episode's report of no discharged obligations at call 40 was not a measure of how far it had to go. The planned 13-id read at 40 calls does not run, because it would return the cap. A read at 120 is a different and much larger commitment: one episode at 8.4 million tokens prices 26 episodes at roughly 218 million, and that figure is a lower bound, since the episode it is priced from passed at 79 of its 120 calls on the task the higher tier found easiest of the three. The two-tier comparison is therefore open and belongs to version 2. What the pair does establish is narrower, and it is one episode against one: on this task the lower tier reaches the same referee verdict as the higher one, and pays 8.7 times the tokens and 3.4 times the calls to get there.
2. **A CLEVER variant with the oracle repaired.** Replace the isomorphism obligation with an implication-with-witness (the agent's specification implies the human one, and the reference implementation satisfies the agent's), exclude problem 18, and rerun the seven problems both arms failed. This changes what the benchmark measures and is registered as a variant, not as a correction to the published Task 1 numbers.

3. **The systems population, with the four things it is missing.** The custom population of Section 2 exists, is frozen, and has produced the cost reading of Section 4. Four registered gaps stood between that reading and a capability comparison on the same population, and each is a separate piece of work; two have since closed and are kept below rather than deleted, because a gap that closed is part of the record of what the design owed. First, and this gap has closed post hoc rather than by registration: a correctness instrument, a referee verdict per cell on the withheld suite, written to a path the scorer reads, so that a premium can be paired with an outcome rather than with a price alone. Section 4.1 reports that pass. It was specified before it ran and after the cost result was known, it did not separate the arms, and a premium on this run can now be set beside an outcome without that outcome being a pre-registered one. Second, also closed, by amendment: three plain-arm cells on `FreeList`, `LRU` and `Paxos` so that all five problems reach $n = 3$ within one run, with the sign test reported both with and without the borrowed cells; if the two readings agree the dependency is discharged, and if they diverge the divergence is the result and is reported ahead of the headline. The cells were fired under AMENDMENT 26 and both readings are reported in Section 4, where both halves of that rule turn out to fire at once. Third, a `## Statement` section authored for the four cards that lack one, which is the only thing standing between the gold pair and $k = 5$; it is an author's hour per card and it decides what the treatment arm measures, so it is not the harness's to invent. Fourth, an arm carrying the method as written rather than the dieted rendering, which the current cost cap refused once already. A scout for a second population over a contamination-free Rust software-engineering set returned no: of its 113 hand-authored tasks 5 are Rust, below the registered threshold of 8 before any screening, and none of the 5 survived the screen for a specification layer. Version 2's populations ship in the Harbor task format [6], so that the referee, the fence and the task remain one object.

The failure surface on S2-Lean is also a finding about the benchmark: four of the twelve remaining stage-C oracles are permissive on regions their tests never visit, and the four-label triage taxonomy has no cell for a reference specification that contradicts its own tests. The reproduction scripts for both are in the repository, for the benchmark's authors; the disclosure itself is a separate act and is not claimed here.

8 Reproducibility

Everything in this paper can be re-derived, and this section says from what. Every pin is a line in `harness/HASHES.txt`: the CLEVER commit, the Lean toolchain and mathlib revision, the project manifest and lakefile hashes, the Verus release and its rust channel, the z3 [4], vstd and lynette hashes, the resource limit and seed, every prompt, arm file, checker and driver, and the set-hash of the 207 rebuilt Verus views. The evaluation images for the SWE-bench substrate are pinned by digest in `IMAGE-DIGESTS.json`. The episode script refuses to run a stage whose view or checker hash is not the pinned one, and each manifest records the freeze commit, the hashes it asserted, the model id read from every response, and the transcript's hash. The morning-line instruments that produced every rate in this paper are in the repository with their self-tests, and the evidence directories carry their outputs byte for byte.

The systems population is pinned differently, because it is authored here rather than derived from a public source. Its five problems are frozen in a dated export, the scored set is declared by cell identifier rather than by a pattern over the archive, each cell's control directory records

its task, arm and card extras, and each cell's price is read from its own harvested meter file. The client is pinned by absolute versioned path in the registration, not resolved from the shell path. One gap in that chain is reported rather than closed: the matrix's cells carry no build-provenance record of their own, so the mapping from cell to instrument set is a reconstruction after the fact and not a receipt, and the claim that this matrix and the earlier stage ran the same client rests on a byte-size comparison against the earlier stage's own record.

The task populations are re-derived from the pinned sources by the harness's view builders; the repository redistributes only what `PROVENANCE.md` lists. The run records and the S2 episode archives are a separate data asset; its DOI is assigned at release (Zenodo) and recorded in the repository at release. Version 2's populations ship as Harbor tasks so that the referee, the fence and the task are one object.

The code is released under Apache-2.0 and the data and documents under CC BY 4.0. The agent was Claude Code [1] on a consumer subscription, and the transcripts are published as this campaign's own measurements: the provider's Consumer Terms effective 2025-10-08 [2] assign outputs to the user and do not restrict their publication, and the training restriction in the Usage Policy dated 2025-09-15 [3] binds the subscriber rather than a recipient of these files. Both documents are cited at the version read, because both are revised in place.

Acknowledgements

CLEVER is by the Trishul group at UT Austin (MIT licence); VeruSAGE-Bench and lynette are by Microsoft (MIT licence); SWE-bench Verified is by the SWE-bench authors with OpenAI's verification pass; the source repositories of the drawn issues carry their own licences, listed in `PROVENANCE.md`. The five systems components of S3, their withheld suites and their mutant sets were authored for this benchmark by the author's assistant heads under the author's direction, which is the independence limitation stated in Section 4. The runs were executed by Claude Code with Anthropic's Sonnet 5 and Opus 5 models. The protocol documents, amendments and result files in the repository were written by the author's assistant heads under the author's direction and carry the author's responsibility.

References

[1] Anthropic. *Claude Code*, version 2.1.251. `https://code.claude.com/docs/en/overview`.

[2] Anthropic. *Consumer Terms of Service*, effective October 8, 2025. `https://www.anthropic.com/legal/consumer-terms`.

[3] Anthropic. *Usage Policy*, effective September 15, 2025. `https://www.anthropic.com/legal/aup`.

[4] Leonardo de Moura and Nikolaj Bjørner. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008)*, pages 337–340, 2008. `https://doi.org/10.1007/978-3-540-78800-3_24`.

[5] Leonardo de Moura and Sebastian Ullrich. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction — CADE 28*, pages 625–635, 2021. `https://doi.org/10.1007/978-3-030-79876-5_37`.

[6] Harbor Framework Team. *Harbor: A framework for evaluating and optimizing agents and models in container environments*. Zenodo, 2026. https://doi.org/10.5281/zenodo.20953922.

[7] Jason Hickey. AI with Authority, from Application to Silicon. arXiv:2608.21356, 2026. https://arxiv.org/abs/2608.21356.

[8] Jason Hickey. The Salt method: canonical definition. docs/SALT-METHOD.md in https://github.com/jyh/salt, commit a8eac8bc, 2026. Apache-2.0.

[9] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770. https://arxiv.org/abs/2310.06770.

[10] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying Rust Programs using Linear Ghost Types. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):286–315, 2023. https://doi.org/10.1145/3586037.

[11] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, 2020. https://doi.org/10.1145/3372885.3373824.

[12] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. https://doi.org/10.1073/pnas.1708274114.

[13] Princeton NLP. *SWE-bench Verified*. Hugging Face dataset, split test, 500 rows; revision c104f840 as pinned here. The dataset card carries no licence field. https://huggingface.co/datasets/princeton-nlp/SWE-bench_Verified.

[14] Alessandro Sosso, Akhil Arora, and Bas Spitters. Agentic Proving for Program Verification. arXiv:2605.23772, 2026. https://arxiv.org/abs/2605.23772.

[15] Xudong Sun, Wenjie Ma, Jiawei Tyler Gu, Zicheng Ma, Tej Chajed, Jon Howell, Andrea Lattuada, Oded Padon, Lalith Suresh, Adriana Szekeres, and Tianyin Xu. Anvil: Verifying Liveness of Cluster Management Controllers. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*, pages 649–666. USENIX Association, 2024. https://www.usenix.org/conference/osdi24/presentation/sun-xudong.

[16] Amitayush Thakur, Jasper Lee, George Tsoukalas, Meghana Sistla, Matthew Zhao, Stefan Zetzsche, Greg Durrett, Yisong Yue, and Swarat Chaudhuri. CLEVER: A Curated Benchmark for Formally Verified Code Generation. arXiv:2505.13938, 2025. https://arxiv.org/abs/2505.13938.

[17] Chenyuan Yang, Natalie Neamtu, Chris Hawblitzel, Jacob R. Lorch, and Shan Lu. VeruSAGE: A Study of Agent-Based Verification for Rust Systems. arXiv:2512.18436, 2025. https://arxiv.org/abs/2512.18436.

A S2-Lean U15, per problem

problem	a0 S	a1 S	a2 S	a0 O	a2 O	a1 O
73	F	P	F	P	P	P
0	F	F	F	F	F	F
146	P	P	P	P	P	P
16	P	P	P	P	P	P
4	P	F	P	F	P	F
38	P	P	P	P	P	P
142	P	P	P	P	P	P
96	F	F	F	F	F	F
112	F	F	F	P	F†	F
141	F	F	F	P	P	P
31	P	P	P	P	P	P
54	P	P	P	P	P	P
127	F	F	F	F	F	F
18	F	F	F	F	F	F
74	P	P	P	P	P	P
proven	8	8	8	10	10	9

S = `claude-sonnet-5`, O = `claude-opus-5`, stage B, 100-call cap. † refused by the pragma screen; a registered re-check over the frozen artifact found a complete proof (diagnostic 11 of 15).

B The stage-B failure triage

problem	label	ground, in one line
0	HUMAN-LOOSE	conclusion guarded by `numbers.length > 1`; vacuous on short lists; witness recorded
4	HUMAN-LOOSE	guarded by `0 < numbers.length`; vacuous on `[]`
18	AGENT-WRONG	agents scan `List.range (|s| - |t| + 1)`, contradicting a `#test`; the human spec is also unsatisfiable
73	PROOF-HARD	equivalent specs; the equivalence needs an optimality argument
96	HUMAN-LOOSE	spec fixes membership only, never order or multiplicity; witness `primes ++ primes`
112	PROOF-HARD	true isomorphism; one sibling cell proved it completely
127	HUMAN-LOOSE	guarded by interval well-formedness; vacuous on ill-formed intervals
141	PROOF-HARD	reduces to a `String.splitOn` equivalence; 7 to 8 kB of lemmas written and still out of rounds

Totals over 30 triageable cells: HUMAN-LOOSE 17, AGENT-WRONG 5, PROOF-HARD 8; one screen-refused cell excluded.

C The S2-Rust hard band

Upper tercile of the 180 live tasks by reference-proof wall, rank ≥ 120; band $n = 60$, Anvil-Advanced 42 and NRKernel 18; band wall median 4.55 s, max 358.60 s; band bytes median 165,568.

Draw of 13 at seed 20260902: 11 Anvil-Advanced, 2 NRKernel. Ten run under the sequential stop: 9 `PASS`, 1 `VERIFY_FAIL` at the cap; sighted episodes reach the first clean referee run at a median of 2 referee calls (p90 9). The 9 is a lower bound on the band's pass count at this cap: the failed episode ended at 40 of 40 calls one obligation short (8 verified, 1 error), the uncensored passing call counts run 23 to 39 against the cap of 40 with p90 = 39, and one pass landed at exactly 40 of 40.

D The systems matrix, per cell

Every metered price in the declared set of Section 4, by condition, as the scorer reports them. The scoreboard prints a premium and not the two medians behind it, because the instrument that computes the two readings prints per-cell prices and the premium and does not print the median; this table is what a reader needs to recover any median in that scoreboard. Prices are US dollars of metered subscription spend per cell, each read from that cell's harvested `METER.txt`.

Reading A is every row as printed. Reading B is every row with the underlined cell removed: those three are the borrowed smoke cells, one each on the three problems the matrix root could not take to $n = 3$ on its own. Three conditions therefore stand at $n = 4$ under reading A and at $n = 3$ under reading B, and the rest are $n = 3$ under both. At $n = 4$ the median is the mean of the middle pair and is not any cell's price, which is why the three premiums that move between the readings are exactly the three conditions listed with four cells.

problem	arm	cells
Crc32	plain-bare	$5.36 $6.21 $7.64
Crc32	diet-bare	$6.19 $7.21 $7.63
FreeList	plain-bare	$13.02 $23.38 $13.77 $13.01
FreeList	diet-bare	$37.95 $37.60 $35.41
LRU	plain-bare	$7.75 $9.73 $6.68 $9.87
LRU	diet-bare	$11.19 $11.21 $14.63
LZW	plain-bare	$8.15 $20.95 $13.95
LZW	plain-statement	$10.24 $11.39 $9.82
LZW	diet-bare	$31.70 $19.18 $18.16
LZW	diet-statement	$22.53 $15.34 $18.54
Paxos	plain-bare	$9.49 $14.20 $16.78 $20.16
Paxos	diet-bare	$37.93 $37.65 $23.52

Four cells of the matrix root are refused by the scorer rather than defaulted, because their cost line does not open with a bare price: a void meter carries a zero in its prose, and a scanner that reads on would price an unmetered cell at nothing. They are excluded from every reading.